TechZone — Software Test Plan

Team 9 · Revision 2.0 · 11 November 2026 · Part 3 Testing & QA

1. Purpose and document control

Validate the delivered internal store against the revised SRS and SAD, retaining traceability to the originally submitted Part III Test Plan. All 65 original unique IDs remain; TC-PUR-06 is added for the omitted receipt-recording requirement FR-PUR-006, making 66 cases. The original repeated TC-SEC-03 counts once. No faculty approval of deviations or unmet baseline scope is claimed.

This plan replaces the original technology/routes/procedures for the current delivery. It does not convert an absent feature into a pass. Case results distinguish PASS, PARTIAL, DEFERRED, N/A and PENDING. The original PDFs remain history.

2. Scope and test items

Authentication, server permissions, products/categories, inventory/search/adjustment/low stock, suppliers, purchase orders/partial/full receipts, sales, reports, common messages/integrity, security snapshot review and measurable local NFRs. Tests include unit/API-client, service, HTTP/database integration, React/jsdom component and ordered workflow checks. Real-browser automation is separate evidence.

Unimplemented baseline items are explicitly tracked: FR-AUTH-006 account mutations; FR-INV-006 history filters; FR-REP-003 dedicated aggregate purchase report. Production HTTPS, least-privilege deployed DB credentials and 99% observed availability cannot be validated against the local-only deployment. Firefox and Edge checks remain pending.

3. Strategy and levels

Level / method	Actual tools and coverage
Unit and deterministic	Node built-in test runner, assert and mocks; permission matrix, API adapter/errors, validation helpers

contracts	
Service/database integration	Real MongoDB validators/indexes/transactions; source/actor integrity; forced rollback and retries
HTTP/system integration	Supertest against createApp; authenticated role matrix, valid/invalid input, concurrent requests
Frontend components	React act, jsdom and Vite SSR; forms, role visibility, filter/clear, errors and saved-write retries
End-to-end API	12 ordered store steps in server/tests/end-to-end.test.js
Browser workflow	Installed Chromium/ChromeDriver via scripts/qa-browser.mjs; actual UI and API with temporary data
Performance and volume	100 warmed sequential authenticated reads; 10000 products and 100000 schema-validated audit records
Static/snapshot review	ESLint, Vite build, Git whitespace checks, source module boundaries, client secret inspection

No new test/coverage package is introduced. Node coverage instruments imported code visible to its runtime; Vite SSR JSX does not yield a whole-client coverage result. Reports identify every unmeasured source file. Backend startup is outside the imported test scope. Tests are not called purely isolated units when they use a database or browser.

4. Environment, data and entry criteria

Node >=24, npm dependencies installed from lockfile and reachable MongoDB replica set. Set TEST_MONGODB_URI to a dedicated test environment. Test files always override its database name with unique `techzone_test_*` databases and drop only those databases. Never point tests at application data. This execution used Arch Linux, Node 26.12.1 and MongoDB 8.3.9, with isolated QA replica set techzone-qa at 127.0.0.1:27019.

Fixtures use synthetic staff/product/supplier records and generated passwords; no real customer data. Exact money boundaries, negative/fractional/unsafe quantities, inactive references, inclusive date boundaries, empty results and duplicate retries are represented. Fault injection controls second-line audit writes, receipt persistence and transient transaction retry failures. Raw state before/after rejection/rollback is compared in consistency tests.

The normal response-time workload is 20 reads each for products, inventory, suppliers, purchases and inventory reports, after warm-up, concurrency 1, a small local fixture,

excluding login/hash and public network time. Require at least 95/100 within 2000 ms. The volume workload uses 10000 products/stock rows and ten +1 adjustment events per product (100000 total), reconciling to 10 units per product. Search, full inventory report and stock adjustment each have a documented local 5000 ms usability budget; this does not replace the separate response-time requirement or assert concurrent production scale.

5. Execution procedure and deliverables

From an up-to-date checkout, start the project replica set (`npm run db:start`) or use a separately configured test replica set. Existing dependencies and generated frontend assets are not committed. Execute:

```
npm run test:coverage
node scripts/qa-browser.mjs
node scripts/qa-review.mjs
node scripts/testing-reports.mjs
```

The browser script requires installed `chromedriver` and Chromium (default `/usr/bin/chromium`, overridable with QA_BROWSER_BINARY), free ports 5001/5174/9516 and loopback access. It creates a disposable seeded database, runs UI actions, deletes the browser session, closes servers and drops that database. Other browsers require the manual matrix in the system validation report. qa-review runs lint/build/whitespace checks and a bounded current client-secret/module review.

Reports: Unit Test, System Validation, Code Coverage, Branch Coverage, and consolidated cases. Raw JSON, source hashes and LCOV reside in docs/testing/evidence. Report generation validates labelled automated PASS rows against passing tests and checks case uniqueness. The four area files are the editable case source; regenerate the combined table after edits.

6. Roles, schedule and review

Owner	Case source / responsibility	Reviewer
Annamalai	auth-security.md; review authentication/security mapping and final report package	Atharv
Apurv	products-stock.md; products/stock/atomicity/messages/business rules	Avaneesh

Avaneesh	inventory-sales-reports.md; listing/sales/reports	Apurv
Atharv	purchases-common-nfr.md; purchases/common/NFR; integrated revised documents	Annamalai

These are delivery assignments, not assertions that reviews occurred. Atharv initially requested a direct-main push as AtharvMitt. GitHub rejected the push with GH006 because main requires a pull request. Atharv subsequently authorized a QA branch and PR as AtharvMitt. Peer review and protected-main merge remain outstanding; no approval is fabricated.

Original Part 3 due date: 6 November 2026. Catch-up target: 12 November 2026 EOD IST. Run and document local QA on 11 November; teammate review, required browser matrix and document decisions by target EOD. Atharv owns uploading the already-recorded one-minute Sprint 1 video and recording/uploading the two-minute Sprint 2 video later. No video upload or recording is claimed here.

7. Pass/fail, exit, suspension and resumption criteria

An automated case passes only when its documented assertions pass in the recorded run. A review/manual case needs observed evidence, environment and steps. N/A requires a reason and scope; PARTIAL and DEFERRED retain baseline gaps. There is no invented minimum coverage percentage: report observed line/function/branch counts, uncovered paths and unmeasured files honestly.

Exit requires zero failed/cancelled/skipped required automated cases, successful lint/build, all 66 dispositions with evidence, four reports/raw coverage, updated SRS/SAD/STP, peer review and completion or explicit team acceptance of outstanding manual/scope issues. Local automation success alone does not satisfy full baseline acceptance. Suspend affected execution if replica-set setup, fixture isolation or environment fails; preserve diagnostics and rerun after correction. Do not mark a blocked/unrun scenario PASS.

8. Defect management and risk

Record requirement/TC, reproduction, environment, expected/actual behavior, severity, owner, fix and retest. Critical stock/integrity/authorization failures block demo acceptance. Known prior sale retry defect and verified fix are retained in docs/stock-consistency.md. This run's baseline scope gaps are listed in the system report; no issue closure, teammate approval or faculty waiver is inferred. A short local NFR sample and Chromium alone do not establish production performance/availability or all-browser support.

9. Full revised case inventory

The table below is generated from the four owner area files. Each row contains the original requirement, actual behavior, deviation, executable evidence/procedure and current result. The execution report additionally lists per-use-case results and the pending manual matrix.

TC ID	Requirement	What we test now	Change from original plan	Automated test / manual steps	Result
TC-AUTH-01	FR-AUTH-001	Active user logs in and receives a session/user role.	Cookie session via POST /api/auth/login; React workspace instead of planned Django dashboard.	server/tests/auth.test.js; scripts/qa-browser.mjs	PASS (automated)
TC-AUTH-02	FR-AUTH-002	Wrong password and unknown email both return 401.	Express JSON error and no authenticated session.	server/tests/auth.test.js	PASS (automated)
TC-AUTH-03	FR-AUTH-003 / SEC-009	Inactive login and subsequent deactivation deny access.	Mongoose user status checked on every session request.	server/tests/auth.test.js	PASS (automated)
TC-AUTH-04	FR-AUTH-004	Logout deletes session; old cookie cannot authorize.	POST /api/auth/logout returns 204.	server/tests/auth.test.js; scripts/qa-browser.mjs	PASS (automated)
TC-AUTH-05	FR-AUTH-005	Stored role is read from the database; role changes apply next request.	Role association tested through model updates; account creation/edit UI/API (FR-AUTH-006) is deferred.	server/tests/permissions.test.js	PASS (automated)
TC-SEC-01	SEC-001	Protected API requests without a session return 401.	HttpOnly cookie and requireAuth middleware.	server/tests/auth.test.js; client/tests/api.test.js	PASS (automated)
TC-SEC-02	SEC-002	Salted scrypt hashes verify passwords; hashes hidden from user output.	Node crypto scrypt replaces Django password handling.	server/tests/schema.test.js; server/tests/auth.test.js	PASS (automated)
TC-SEC-03	SEC-003	Every role/permission and	Express permission guards; five roles.	server/tests/permissions.test.js;	PASS

		Sprint 2 operation is checked by server.		server/tests/permission-regression.test.js	(automated)
TC-SEC-04	SEC-004	Forged client role information cannot raise permissions.	Stored user role is authoritative.	server/tests/permissions.test.js	PASS (automated)
TC-SEC-05	SEC-005	Review production HTTPS scope and secure cookie configuration.	Only loopback HTTP development deployment exists; no TLS deployment to inspect.	server/src/modules/auth/sessions.js; production check requires HTTPS URL, certificate and network inspection.	N/A (production deployment absent)
TC-SEC-06	SEC-006	Malformed types, unknown fields and literal search inputs are rejected or safely handled.	Mongoose/Express validation replaces SQL/Django sanitization.	server/tests/products.test.js; server/tests/suppliers.test.js	PASS (automated)
TC-SEC-07	SEC-007	Inspect tracked files, client source and production assets for database/seed secrets.	Vite client uses relative /api; server-only dotenv; .env ignored.	docs/testing/evidence/security-review.json; scripts/qa-review.mjs	PASS (review)
TC-SEC-08	SEC-008	Unknown/wrong-password logins return the same status/message.	Decoy scrypt hash used for unknown users; timing equivalence is not asserted.	server/tests/auth.test.js	PASS (automated)
TC-SEC-09	SEC-009	Inactive account cannot authenticate.	Same generic 401 as other failed logins.	server/tests/auth.test.js	PASS (automated)
TC-SEC-10	SEC-010	Review least-privilege production database credentials.	Local QA replica set is isolated and has no authentication; production privileges are unverified.	Requires deployed application DB user and privilege inspection.	N/A (production deployment absent)
TC-INV-01	FR-INV-001	Listing shows current persisted quantities after	GET /api/inventory joins MongoDB	server/tests/inventory.test.js	PASS

		changes.	collections.		(automated)
TC-INV-04	FR-INV-004 / BR-003	At/below minimum appears in low-stock filter and report.	Threshold is calculated on each read.	server/tests/inventory.test.js; server/tests/end-to-end.test.js	PASS (automated)
TC-SALE-01	FR-SALE-001	Authorized multi-item sale succeeds.	POST /api/sales uses Idempotency-Key.	server/tests/sales-reports.test.js	PASS (automated)
TC-SALE-02	FR-SALE-002	Zero, negative, fractional, string and unsafe quantities rejected.	Positive safe integers only.	server/tests/sales-reports.test.js	PASS (automated)
TC-SALE-03	FR-SALE-003 / BR-008	Insufficient stock rejects entire sale.	MongoDB transaction rolls back every line.	server/tests/sales-reports.test.js	PASS (automated)
TC-SALE-04	FR-SALE-004 / BR-011	Total equals quantity times stored selling price.	Server snapshots integer minor-unit prices; client totals are not trusted.	server/tests/sales-reports.test.js	PASS (automated)
TC-SALE-05	FR-SALE-005	Sold quantities decrement exactly once.	Sale, stock and audit records commit together.	server/tests/sales-reports.test.js	PASS (automated)
TC-SALE-06	FR-SALE-006	Distinct completed sales have distinct transaction numbers.	SALE- plus object ID, protected by a unique index.	server/tests/sales-reports.test.js	PASS (automated)
TC-SALE-07	FR-SALE-007	Filter sales by transaction number and inclusive dates.	GET /api/sales? transactionNumber=&from=&to=.	server/tests/sales-reports.test.js	PASS (automated)
TC-REP-01	FR-REP-001	Inventory report matches product/category, quantities and status.	GET /api/reports/inventory; optional status/category/stock filters.	server/tests/sales-reports.test.js	PASS (automated)
TC-REP-02	FR-REP-002	Low-stock report includes	GET /api/reports/low-stock.	server/tests/sales-reports.test.js	PASS

		equality and excludes inactive products.			(automated)
TC-REP-03	FR-REP-003	Purchase history supports supplier/date filters and per-order totals.	No dedicated /api/reports/purchases endpoint, aggregate report total or purchase report screen.	server/tests/purchases.test.js (TC-PUR-09); client/tests/purchases-ui.test.js; dedicated reporting remains unimplemented.	PARTIAL (baseline gap)
TC-REP-04	FR-REP-004	Sales report filters dates and sums stored sales.	GET /api/reports/sales; inclusive UTC date boundaries.	server/tests/sales-reports.test.js	PASS (automated)
TC-REP-05	FR-REP-005	Inventory/sales report counts and totals agree with stored transactions.	Purchase aggregate reporting is excluded and remains TC-REP-03 gap.	server/tests/sales-reports.test.js	PASS (automated)
TC-PROD-01	FR-PROD-001	Create product/category and persist zero-stock row atomically.	POST /api/products and /api/categories; MongoDB.	server/tests/products.test.js	PASS (automated)
TC-PROD-02	FR-PROD-002 / BR-001	Normalized duplicate SKU rejected on create/update.	Unique index also reserves inactive SKUs, stricter than original active-only wording.	server/tests/products.test.js	PASS (automated)
TC-PROD-03	FR-PROD-003	Valid product edits persist.	PATCH /api/products/:id; money stored in integer minor units.	server/tests/products.test.js	PASS (automated)
TC-PROD-04	FR-PROD-004	Create/edit categories; reject deleting referenced categories.	Shared categories collection and reference checks.	server/tests/products.test.js	PASS (automated)
TC-PROD-05	FR-PROD-005	Search inventory by product SKU/name/category.	Search lives on GET /api/inventory and Inventory screen; catalogue has no	server/tests/inventory.test.js	PASS (automated)

			search field.		
TC-PROD-06	FR-PROD-006 / BR-009	Deactivation preserves historical records and stock.	DELETE /api/products/:id performs logical retirement.	server/tests/products.test.js	PASS (automated)
TC-INV-02	FR-INV-002	Signed stock adjustment persists product, reason, actor and time.	POST /api/inventory/:productId/adjustments; MongoDB transaction.	server/tests/inventory.test.js	PASS (automated)
TC-INV-03	FR-INV-003 / BR-002	Reject adjustment below zero without history write.	Guarded atomic update replaces relational locking.	server/tests/inventory.test.js	PASS (automated)
TC-INV-05	FR-INV-005	Purchase, sale and adjustment audit events reconcile to stored stock.	History verified in MongoDB; no general history screen.	server/tests/end-to-end.test.js	PASS (automated)
TC-INV-06	FR-INV-006	No general inventory-history API/UI is implemented.	Baseline history filtering by product/date is deferred; receipt history alone is not equivalent.	Code inspection: server/src/modules/inventory/routes.js exposes listing and adjustments only.	DEFERRED (baseline gap)
TC-NFR-08	NFR-009	Force later-line audit/receipt failures and verify full rollback.	Real MongoDB multi-document transactions on a replica set.	server/tests/stock-consistency.test.js	PASS (automated)
TC-NFR-09	NFR-010	Invalid receipt quantities identify the corrective action; saved writes distinguish refresh errors.	React alert text and shared JSON messages replace planned field framework.	client/tests/purchases-ui.test.js	PASS (automated)
TC-BR-03	BR-003	Quantity equal to minimum is low stock; above minimum is	Computed from availableQty <=	server/tests/inventory.test.js	PASS

		not.	minimumStock.		(automated)
TC-BR-08	BR-008	Insufficient sale stock leaves sale/history/stock unchanged.	Guarded decrement in sale transaction.	server/tests/sales-reports.test.js	PASS (automated)
TC-PUR-01	FR-PUR-001	Create/edit supplier contacts; persist normalized fields.	Name required; optional contact, phone, email and address; /api/suppliers.	server/tests/suppliers.test.js	PASS (automated)
TC-PUR-02	FR-PUR-002 / BR-009	Deactivate supplier without deleting orders or snapshots.	DELETE is logical deactivation; existing orders may still receive goods.	server/tests/purchases.test.js; server/tests/suppliers.test.js	PASS (automated)
TC-PUR-03	FR-PUR-003 / BR-004	Create order for active supplier and products without changing stock.	POST /api/purchases with Idempotency-Key; DRAFT/ORDERED states.	server/tests/purchases.test.js	PASS (automated)
TC-PUR-04	FR-PUR-004 / BR-005	Reject missing/empty/invalid lines and references.	Positive safe whole quantities; at most 100 unique products.	server/tests/purchases.test.js	PASS (automated)
TC-PUR-05	FR-PUR-005 / BR-011	Derive exact order totals from quantities and unit costs.	Integer minor units replace decimal database money.	server/tests/purchases.test.js	PASS (automated)
TC-PUR-06	FR-PUR-006	Record partial/full received quantities against an order.	Added missing TC-PUR-06: absent from original 65 IDs but required by SRS and assignment.	server/tests/purchases.test.js	PASS (automated)
TC-PUR-07	FR-PUR-007 / BR-006	Receipt increments only intended stock and appends source/actor history.	POST /api/purchases/:id/receipts transaction; ordering alone leaves stock unchanged.	server/tests/purchases.test.js	PASS (automated)
TC-PUR-08	FR-PUR-008	Reject over-receipt, including	No approved exception process exists;	server/tests/purchases.test.js	PASS

		competing requests.	over-receipt always rejected.		(automated)
TC-PUR-09	FR-PUR-009	Filter purchase history by supplier/date/status and clear filters.	GET /api/purchases? supplierId=&from=&to=&status=; inclusive UTC dates.	server/tests/purchases.test.js; client/tests/purchases-ui.test.js	PASS (automated)
TC-COM-01	FR-COM-001	Required fields/types rejected before persistence.	Express validators and browser forms; not all errors are inline by field.	server/tests/products.test.js	PASS (automated)
TC-COM-02	FR-COM-002	Invalid receipt input and API errors describe correction needed.	React role=alert messages; malformed error bodies have fallback text.	client/tests/purchases-ui.test.js; client/tests/api.test.js	PASS (automated)
TC-COM-03	FR-COM-003	Successful receipt shows confirmation and refreshes stock once.	React status message; committed write plus failed refresh is identified separately.	client/tests/purchases-ui.test.js	PASS (automated)
TC-COM-04	FR-COM-004	No-match filters show explicit empty result and support clear.	Inventory/supplier/sale/report empty states.	client/tests/suppliers-ui.test.js; scripts/qa-browser.mjs	PASS (automated)
TC-COM-05	FR-COM-005	Referenced supplier/product deactivation preserves snapshots and history.	Mongoose service reference checks; MongoDB validators/indexes; no SQL foreign keys.	server/tests/purchases.test.js; server/tests/products.test.js	PASS (automated)
TC-NFR-01	NFR-001	100 warmed authenticated reads across five routes: at least 95% within 2 seconds.	Defined small-data local workload, sequential concurrency 1; excludes login and production network.	server/tests/non-functional.test.js; diagnostics in evidence/server-results.json	PASS (automated)
TC-NFR-02	NFR-002	Review availability evidence for a defined operating	No hosted deployment or agreed observation window; a short local run	Needs uptime monitor, start/end times, planned-maintenance exclusions and downtime	N/A (operating window absent)

		window.	cannot prove 99% availability.	log.	
TC-NFR-03	NFR-004	Concurrent/retried orders, receipts and sales create one committed transaction.	Durable idempotency keys and request fingerprints; uncertain frontend retries reuse keys.	server/tests/purchases.test.js; server/tests/stock-consistency.test.js; client/tests/api.test.js	PASS (automated)
TC-NFR-04	NFR-005	Review separated modules, service transaction ownership and core test files.	Express module directories replace Django apps.	docs/specifications/software-architecture-design.md; scripts/qa-review.mjs	PASS (review)
TC-NFR-05	NFR-006	Real API/service/component suites execute without skipped integration tests.	Node test runner, Supertest and jsdom replace pytest; unique disposable test databases.	evidence/server-results.json; evidence/client-results.json	PASS (suite execution)
TC-NFR-06	NFR-007	Run integrated desktop/mobile-width flow in Chromium; Firefox/Edge checks pending.	Headless browser automation is recorded separately from human checks; other required browsers unavailable.	scripts/qa-browser.mjs; follow manual matrix in system-validation-report.md on Firefox and Edge.	PARTIAL (browser matrix pending)
TC-NFR-07	NFR-008	Load 10000 products, 100000 audit records; search, report and adjust stock.	Synthetic schema-validated fixture; single local client, 5-second usability budget per operation.	server/tests/non-functional.test.js; diagnostics in evidence/server-results.json	PASS (automated)
TC-NFR-10	NFR-011	ESLint, build and whitespace checks; inspect Git workflow.	ESLint/GitHub Actions replace SonarQube/Jenkins; main protection requires the authorized QA PR; peer review is pending.	docs/testing/evidence/quality-checks.json; teammate review still required.	PARTIAL (peer review pending)
TC-NFR-11	NFR-012	Inventory events preserve responsible actor, source, reason and timestamp.	Immutable inventoryTransactions collection; adjustment audit tests check Date values.	server/tests/inventory.test.js; server/tests/end-to-end.test.js	PASS (automated)